Ex no.1 Comparative Study of Low-Code Development

AIM:

To study the concept of **Low-Code Development** and compare it with **Traditional Application Development**, using **Zoho Creator** as a low-code development platform for building a Vehicle Service & Repair Request Management application.

THEORY:

Low-Code Development is a software development approach that enables applications to be developed using **visual development environments, drag-and-drop components, pre-built modules, forms, workflows, and configuration tools**, with minimal manual coding. In this experiment, **Zoho Creator** is used to develop a Vehicle Service & Repair Request Management application. The application contains a vehicle service request form and multiple reports such as List View, Spreadsheet View, Kanban View, Calendar View, Map View, Timeline View, Pivot Table, Chart View, and Summary View.

LOW-CODE DEVELOPMENT

Low-Code Development uses a visual environment where application components can be created and configured without writing large amounts of source code.

In the Vehicle Service & Repair Request Management application, Zoho Creator is used to:

- Create the **Vehicle Services Form** using a visual form builder.
- Add fields such as Owner Name, Vehicle Registration Number, Vehicle Type, Contact Number, Service Type, Odometer Reading, Pickup Address, and Preferred Service Date.
- Configure field properties and validation rules.
- Store submitted information in the application's database.

- Create different reports using predefined report types.

TRADITIONAL APPLICATION DEVELOPMENT

Traditional Application Development requires developers to manually design and implement most components of an application using programming languages, frameworks, databases, and development tools.

For the same Vehicle Service & Repair Request Management application, a traditional approach could involve:

- Designing the frontend using HTML, CSS, JavaScript, or a frontend framework.
- Developing backend APIs using languages and frameworks such as Java, Python, Node.js, or similar technologies.
- Designing and configuring the database using MySQL, PostgreSQL, or another database system.
- Writing SQL queries for storing and retrieving service request information.
- Implementing form validation manually.

COMPARISON TABLE

Parameter	Low-Code Development	Traditional Application Development
Development Approach	Visual development and configuration	Manual programming
Coding Requirement	Minimal coding	Extensive coding
Development Speed	Faster application development	Generally takes more development time
User Interface	Pre-built visual components	Developed manually
Database Integration	Often provided by the platform	Requires manual database integration
Form Creation	Drag-and-drop form builder	Forms are manually designed and programmed
Validation	Configuration-based or built-in rules	Usually implemented through code

Parameter	Low-Code Development	Traditional Application Development
Reports	Predefined report and visualization types	Reports need to be developed manually
Customization	Depends on platform capabilities	Highly customizable
Maintenance	Relatively simple for configuration-based applications	Requires code-level maintenance

RESULT:

Thus, the experiment demonstrates the practical difference between **low-code development and traditional application development** in terms of development effort, coding requirements, customization, and application development speed.